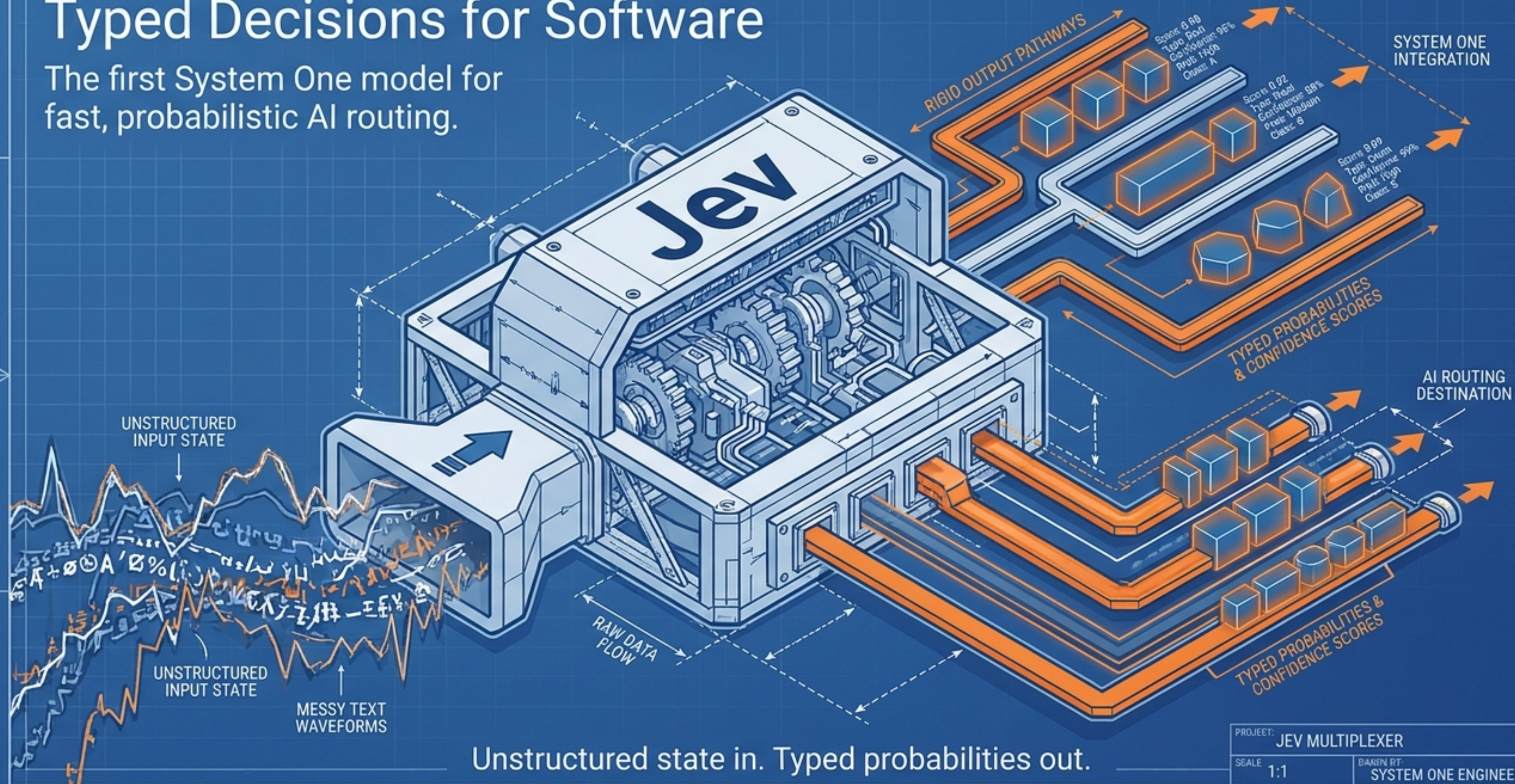


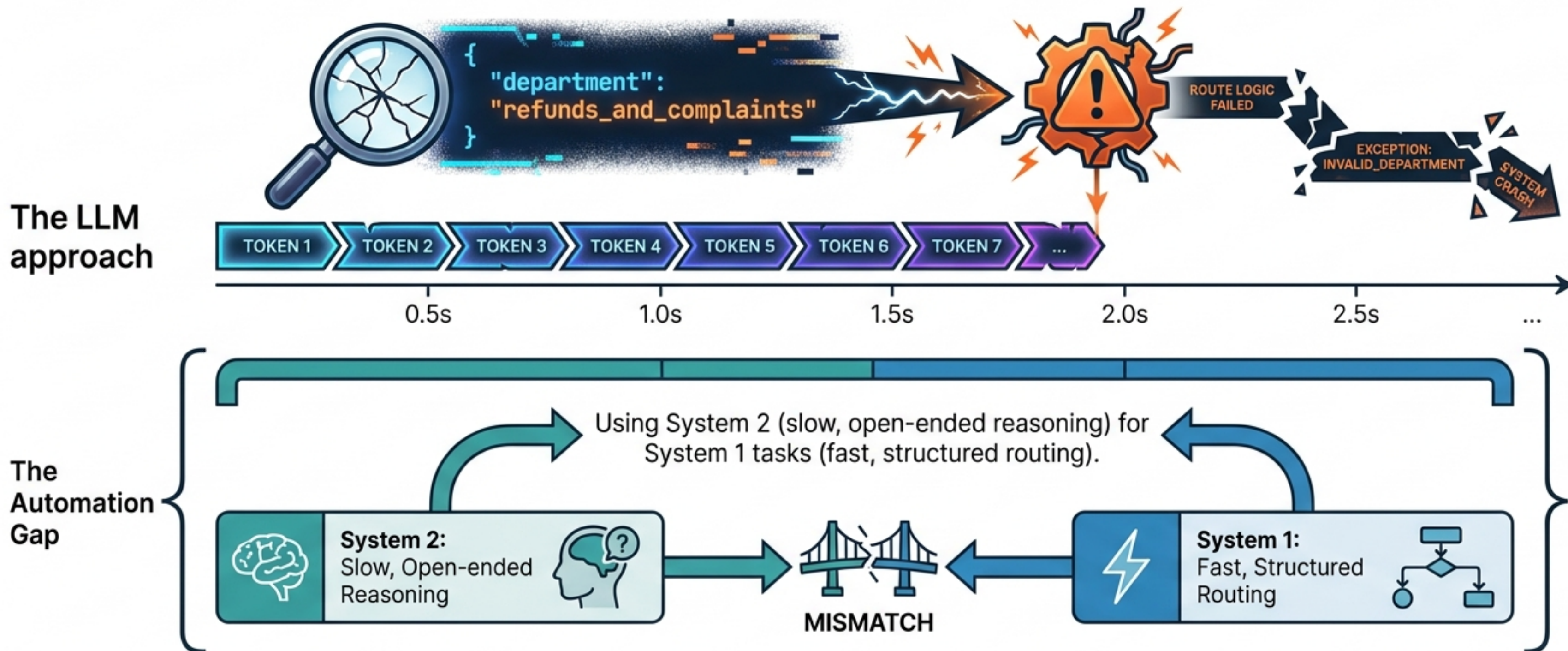
Typed Decisions for Software

The first System One model for fast, probabilistic AI routing.



The brittle reality of LLM agent loops

We force generative models to parse JSON and route logic. The result is software that is **too slow** for real-time user experiences and **too brittle** for deterministic code.



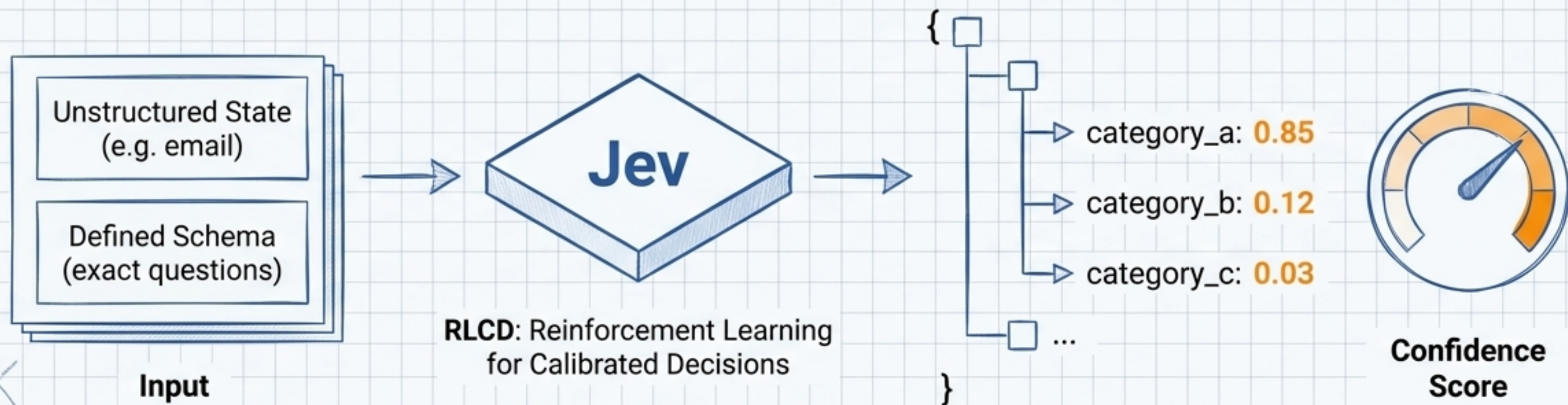
The paradigm shift: Generating text vs. routing probabilities

	LLM (SYSTEM TWO)	JEV (SYSTEM ONE)
Core Action	Autoregressive text generation	Parallel typed answers 
Answer Space	Open-ended replies and prose	Closed, developer-defined schema
Failure Mode	Hallucinated free text outside schema	Strictly bounded options (zero schema hallucinations)
Latency	Seconds + output token delays	~70-500 milliseconds
Cost Structure	Expensive input & output	Input-priced (\$42 per billion tokens), Output free

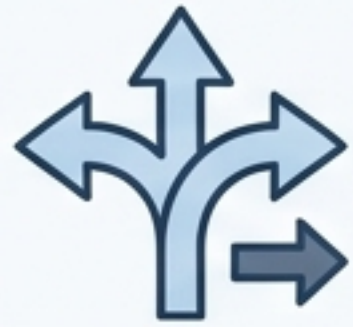
A frontier-intelligence function call

The Cardinal Rule: Jev never writes replies, never generates code, and never explains its reasoning.

It is a **decision layer**. Your software branches on the numbers.



The structural primitives of programmable intuition



CHOICE PRIM-001

Select 1 of N defined options.

[python, javascript, go, other]

-> Outputs selected option + probability distribution

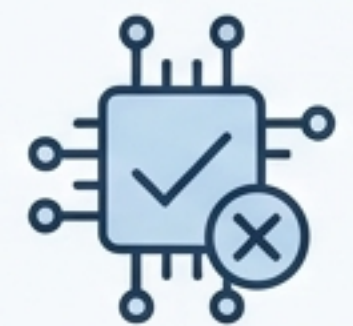


SCORE PRIM-002

Ordered scale with described levels.

[0: calm, 1: frustrated, 2: angry]

-> Outputs numerical score + confidence



NOUL PRIM-003

Boolean statement truth mapping to a probability [0,1].

"Message requests a refund"

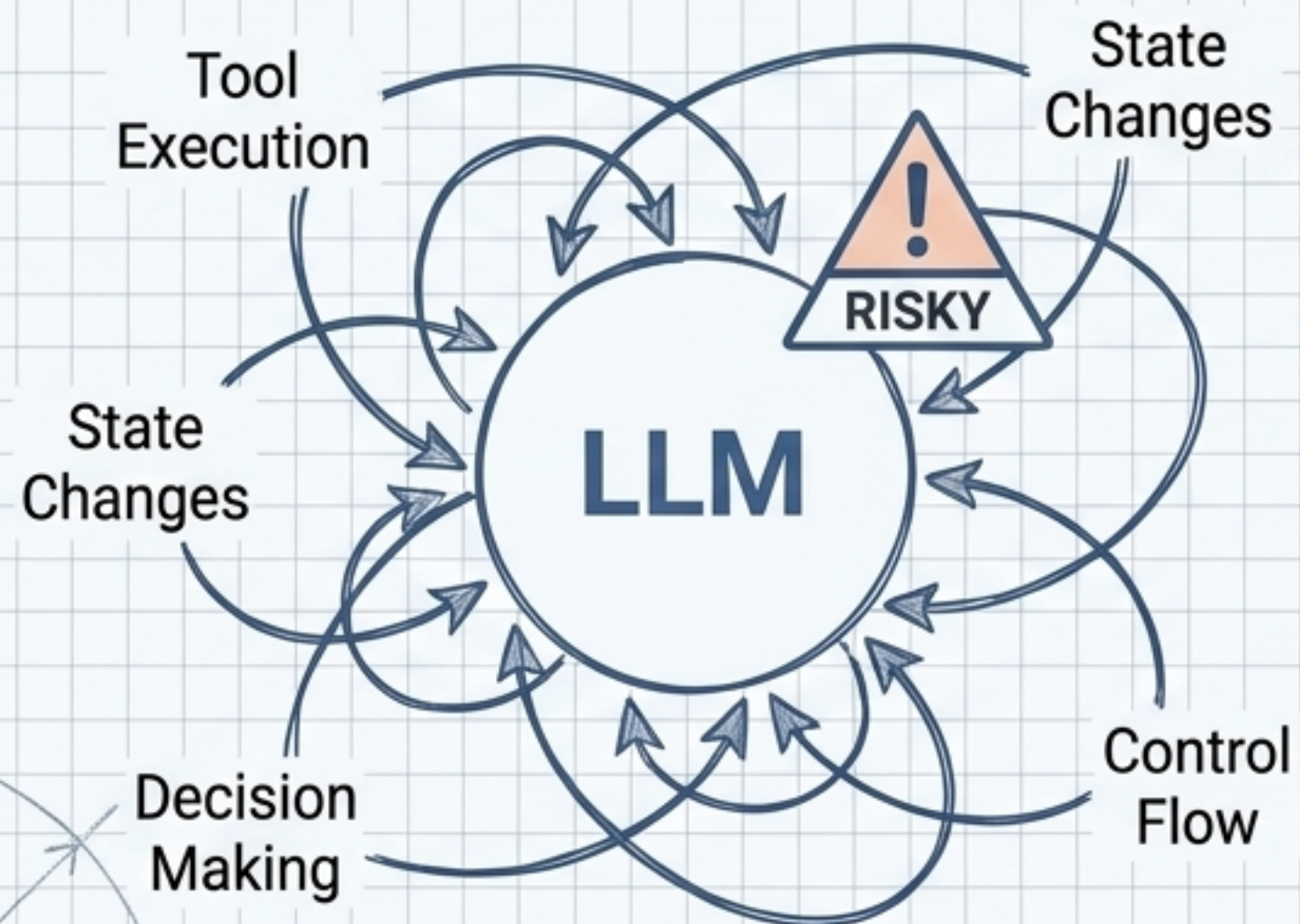
-> Outputs 0.95



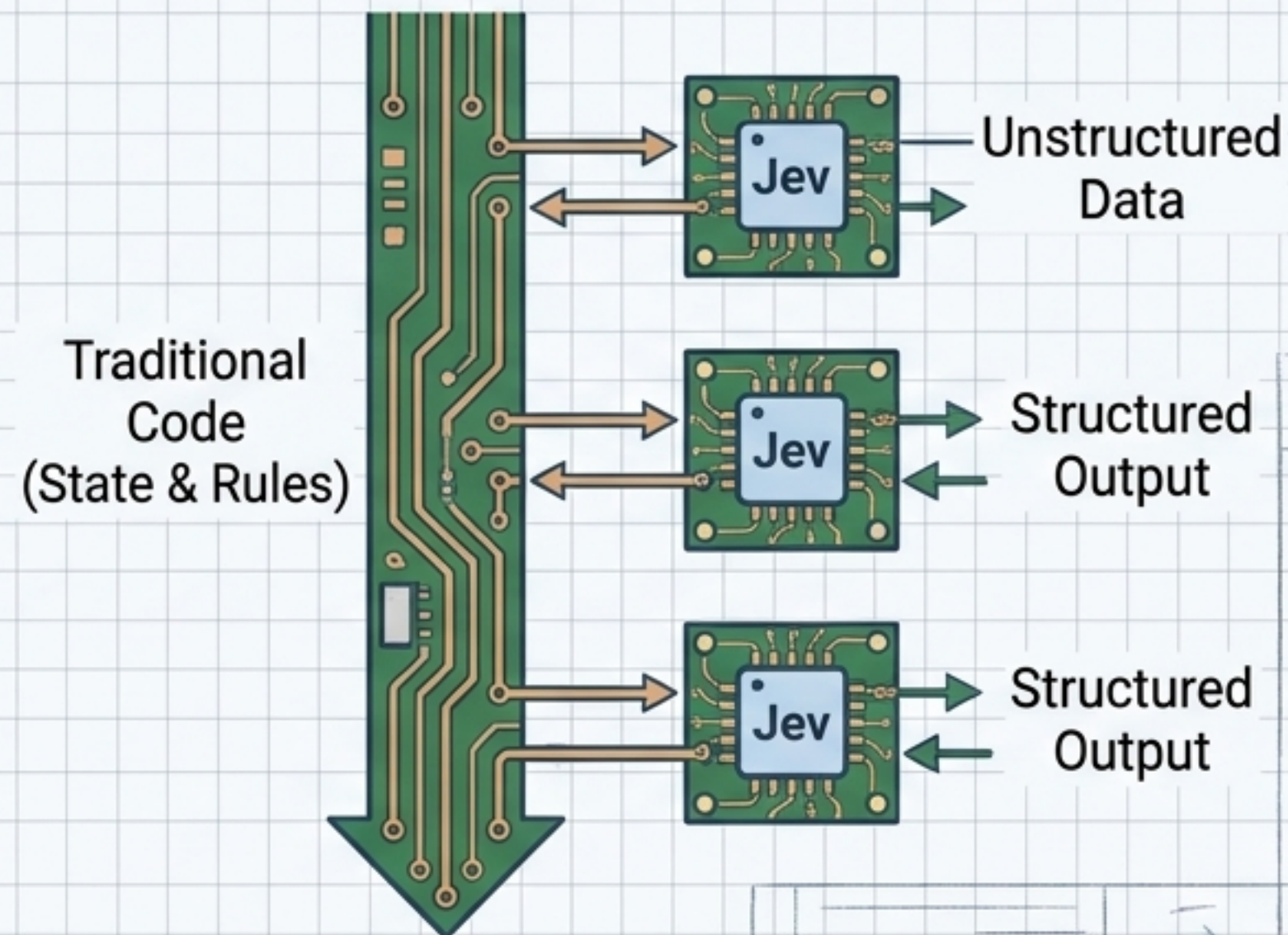
Reclaiming control flow from generative black boxes

Keep control flow, deterministic rules, and side effects in code. Use Jev only where common sense is needed to cross the unstructured-to-structured gap.

LLM Agent While Loop

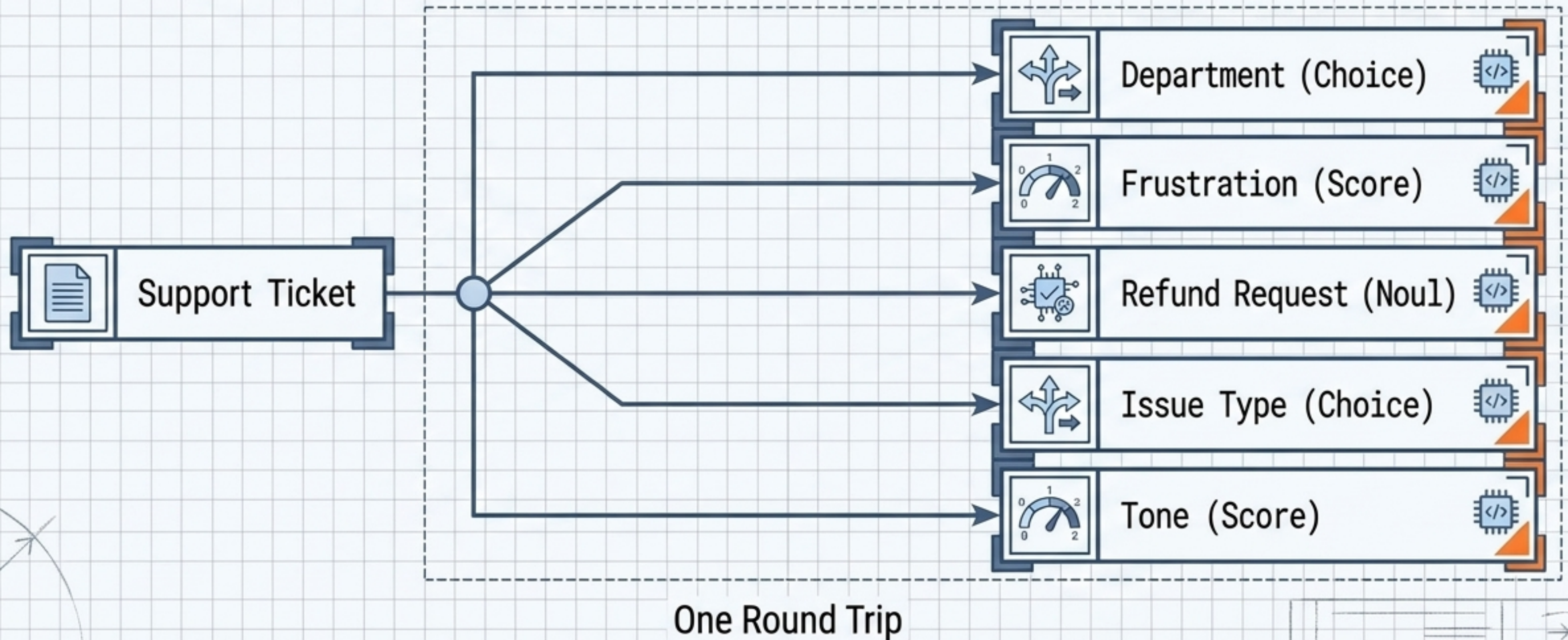


AI-Powered Software



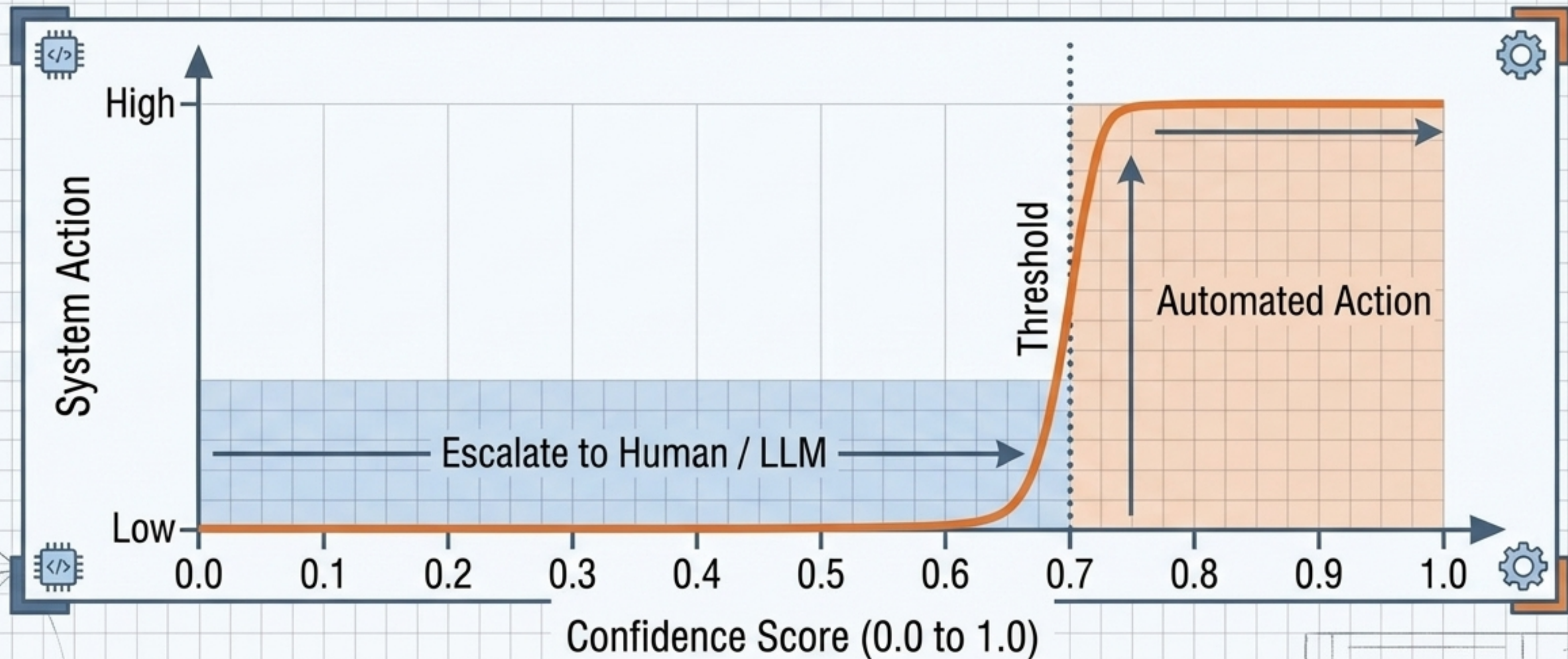
Speculative Fan-Out: Parallel evaluation without the latency penalty

Do not ask one broad question per call. Decompose judgments into narrow, atomic questions. Because Jev evaluates in parallel and output tokens are free, ask every question your code might need simultaneously.



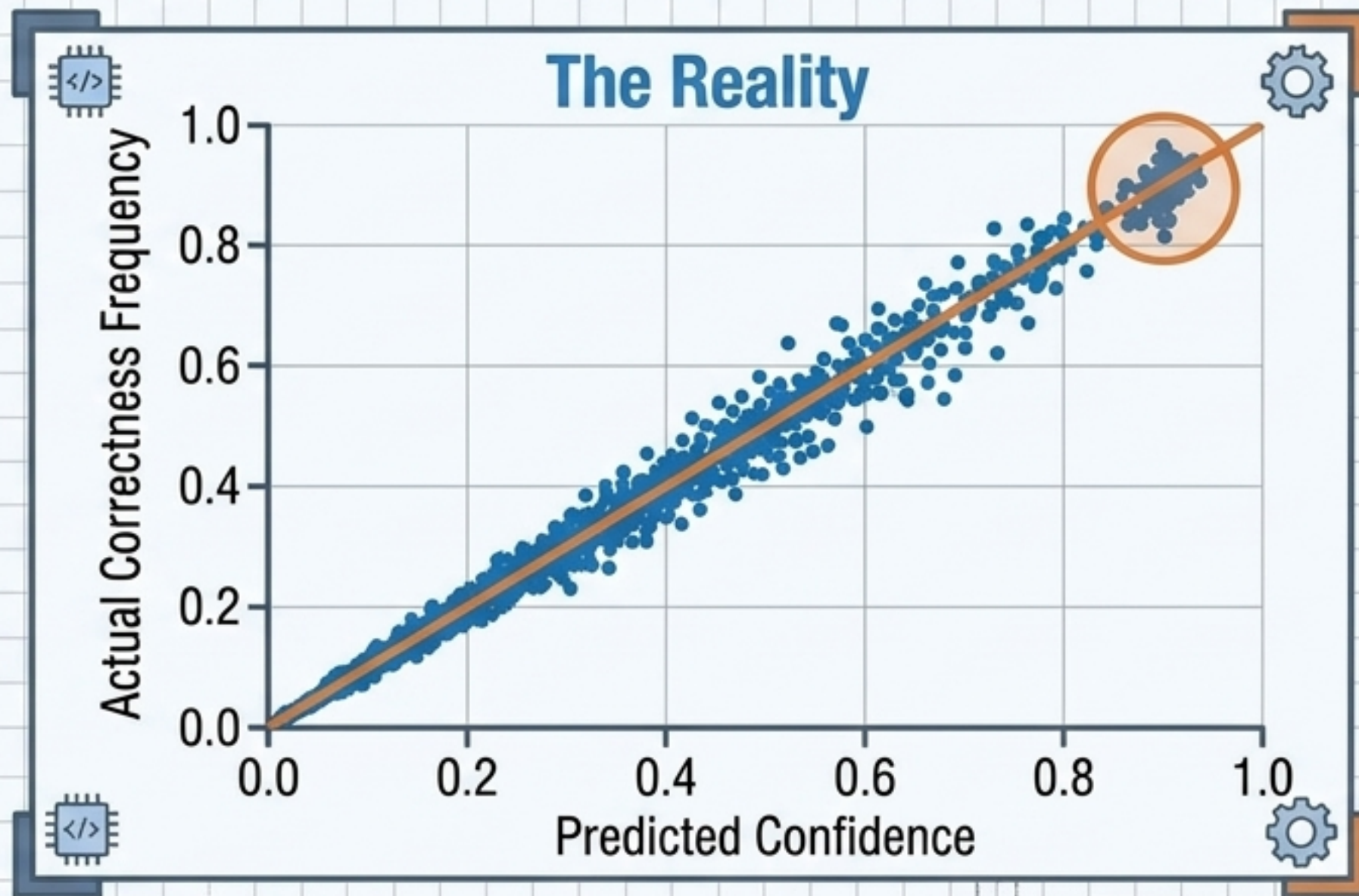
The Confidence Gate: Bounding automation risk

Jev's output is not just an answer; it is a calibrated probability distribution. Low confidence is a feature that triggers safe escalation.



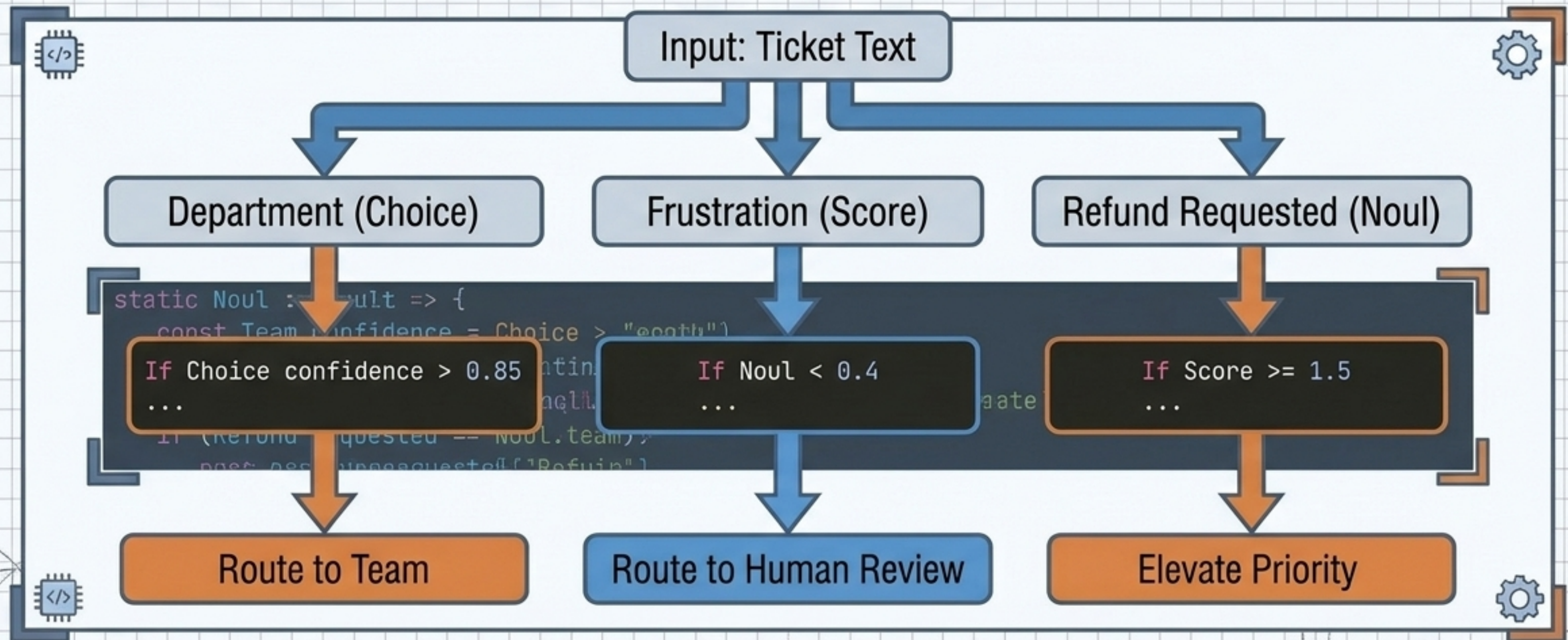
Calibrated uncertainty vs. absolute correctness

A 0.9 confidence score is a calibrated frequency claim across many trials, not a 100% warranty on a single ticket. The aggregate shape is guaranteed; individual answers still require architectural safety nets.



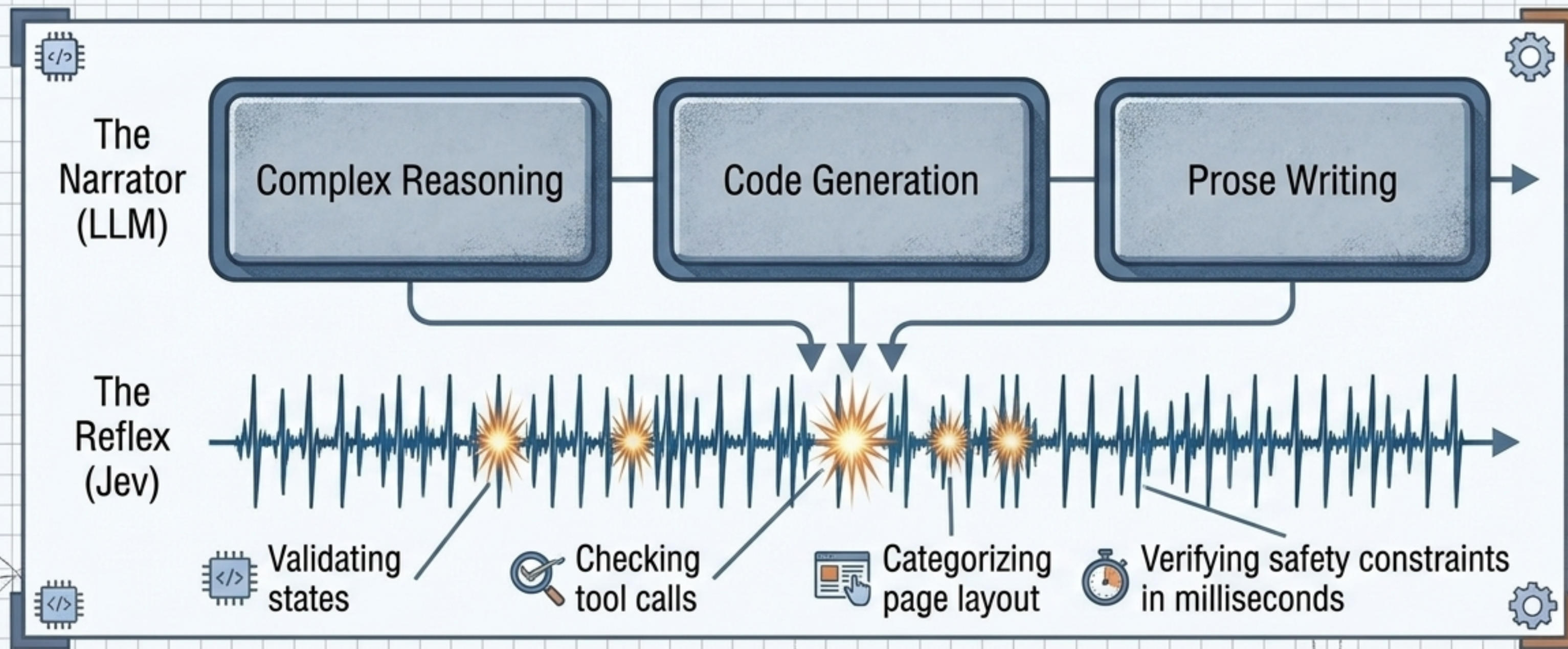
Deconstructing a deterministic support workflow

One request, multiple answers, and routing logic handled entirely by standard IF statements. No parsing hacks required.



The Split Brain: Jev as the agent reflex layer

Offload routine categorization and safety gates to Jev. Reserve slow, expensive LLM tokens exclusively for generating novel prose and complex logic.



The economic mechanics of System One

At this cost and latency, AI classification ceases to be a heavy API call and becomes a microsecond-scale utility function.

70-500ms

End-to-end response latency

\$42.00

Cost per billion input tokens

\$0.00

Cost for output tokens

193x Faster / 444x Cheaper

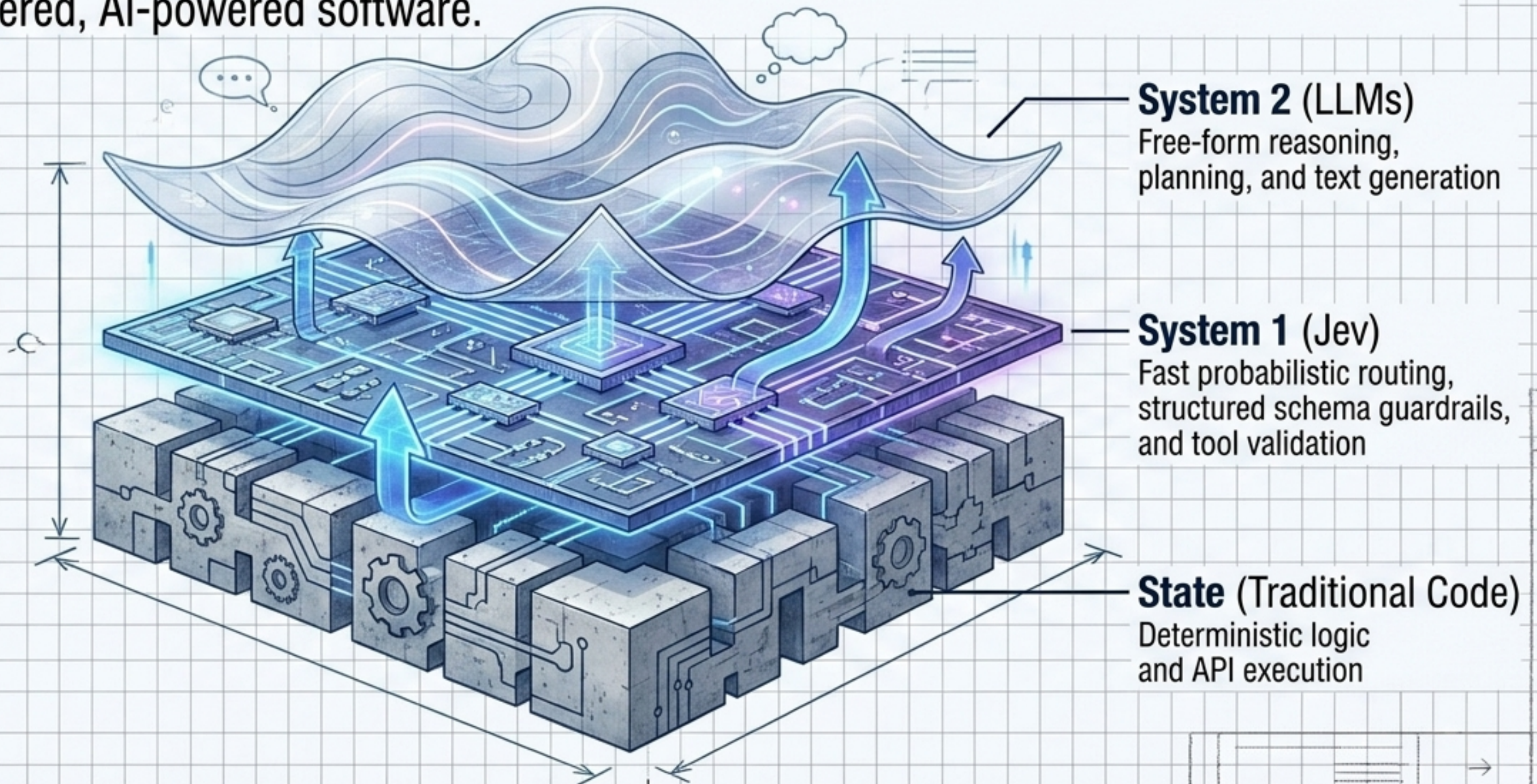
Vendor-reported multipliers vs.
traditional frontier LLMs

Implementation guardrails and common traps

Symptom	Root Cause	Architectural Fix
Unbounded tasks failing	Forcing open-ended generation into a tiny Choice schema	Decompose the state; use LLMs for generative prose
Automation errors on edge cases	Treating Jev as an infallible oracle instead of a calibrated probability	Enforce strict escalation paths based on confidence thresholds
Jev cannot understand the UI	Pushing multimodal images to a text-only API	Pre-process UI into accessibility trees or HTML strings before routing

Synthesizing the modern AI computing stack

AI is no longer a single black box. You don't build autonomous agents; you build multi-layered, AI-powered software.



Identifying the first replacement in your architecture

Read the primitives and build the blueprint at typesafe.ai

